



Alma Mater Studiorum-Università di Bologna

Scuola di Ingegneria

Fondamenti di Informatica T2

Introduzione a JUnit

Corso di Laurea in Ingegneria Informatica

Anno accademico 2023/2024

Prof. ROBERTA CALEGARI

Prof. AMBRA MOLESINI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)



Oltre le `assert`

- Le asserzioni sono un grosso passo avanti rispetto alle print, MA **scrivere molti test basati su `assert` è laborioso e scomodo**
- Per usarle efficacemente occorre **strumentarle**
- Serve un'**infrastruttura** che garantisca:
 - la **notifica** dei problemi in un formato amichevole e analizzabile
 - l'esecuzione di **tutti** i test anche nel caso in cui alcuni **falliscano**
 - adesso, la prima **`assert`** che fallisce causa l'*abort* del programma
 - un **report chiaro e compatto** su quali test siano falliti e quali riusciti
 - da sola, **`assert`** non spiega perché sia fallita (il suo argomento è solo un **`boolean`**, senza messaggi)

JUnit

Cos'è?

- Un **framework di test** scritto esso stesso in Java: una serie di classi gestiscono il lavoro ripetitivo, come
 - far girare i test in batch
 - contare e riportare gli errori e i test falliti, ...

Cosa fa?

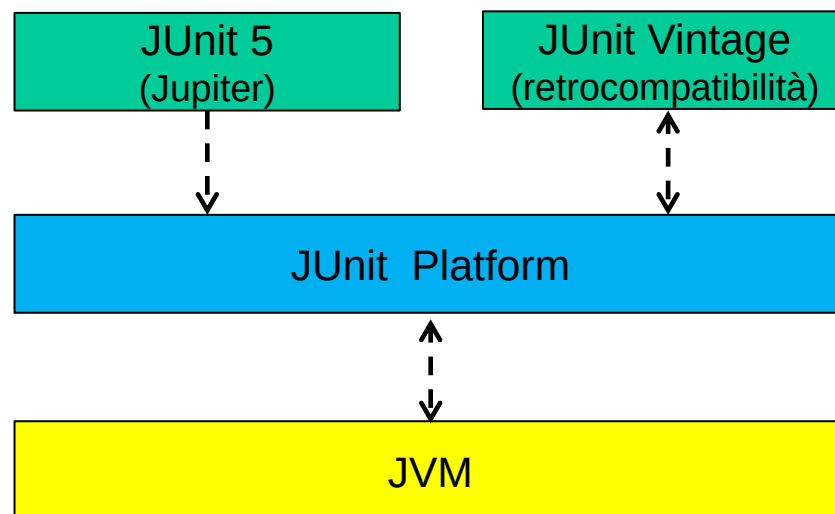
- **Strumenta efficacemente il processo di Unit Testing** (da cui il nome)
- IDEA: verificare il funzionamento di **piccole porzioni (unità) di codice** (un **metodo**, una **classe**, max un **componente**).

Dov'è?

- È integrato all'interno dell'ambiente di sviluppo
- ma funziona anche a linea di comando

JUnit: architettura

- *JUnit Platform* fornisce le basi per lanciare il framework di test
 - definisce le TestEngine API per lo sviluppo di un framework di test
 - fornisce un Launcher da Console per eseguire da riga di comando
 - implementa il Runner che esegue tutti i test
- *JUnit 5 (Jupiter)* è il modello di programmazione attuale
- *JUnit Vintage* è il modulo di retrocompatibilità per vecchi test basati su JUnit 3 e JUnit 4





JUnit: terminologia

- **Test Fixture:** una **classe** i cui metodi *realizzano i casi di test* di un'entità che si vuole raggruppare
 - ATTENZIONE: non tutti i metodi di una *Fixture* realizzano test: possono esserci *altri* metodi con *altri* scopi (è pur sempre una normalissima classe..)
- **Come distinguere i metodi di test dagli altri?**
 - **NON DAL NOME:** vogliamo poter usare i nomi più adatti, a nostra scelta
 - **MA** tramite un **opportuno "tag":** **@Test**
 - *@Test* è un esempio di *annotation*

JUnit: il Runner

- Il **Test Runner** è il motore che fa girare i test
 - esegue TUTTE le test fixtures
 - esegue TUTTI i metodi di test di CIASCUNA test fixture, senza abortire se uno o più test falliscono o «esplodono»
 - produce alla fine un report chiaro e leggibile su «come è andata»
- L'ordine di esecuzione è volutamente *non deterministico*
 - i test devono essere *indipendenti* gli uni dagli altri, ovvero *il risultato di ogni test non deve dipendere dall'ordine di esecuzione*
 - per evitare influenze reciproche, *un buon test non deve dare per scontato che altri metodi funzionino correttamente*
 - ad esempio, il test che verifica l'elaborazione di informazioni lette da un file *non dovrebbe dare per scontato che la lettura da file funzioni*
 - a volte occorre prevedere dei **mock** per simulare altri pezzi del sistema



JUnit: oltre l'assert

- Java in sé prevede l' **istruzione assert**
 - prevede un argomento booleano: se fallisce, il programma abortisce
 - è embedded nel codice di «business logic» (che quindi si «sporca»)
- JUnit la rimpiazza con una serie di **metodi statici assertXXX**
 - JUnit 5: package `org.junit.jupiter.api.Assertions`
 - JUnit 4: vecchio package `org.junit.Assert`
- I metodi **assertXXX**
 - prevedono argomenti *non solo boolean*
 - hanno *nomi più chiari*, che esprimono cosa si intenda collaudare
 - si usano nelle fixture (classi di test), quindi *fuori* dalla «business logic»



da **assert** a **assertXXX**

Metodo	Descrizione
<code>assertEquals(expected, current)</code>	Verifica che i due <i>valori</i> o <i>oggetti</i> siano <i>uguali</i>
<code>assertArrayEquals(expected, current)</code>	Verifica che i due <i>array</i> siano <i>uguali</i>
<code>assertTrue(condition)</code>	Verifica che la condizione sia vera
<code>assertFalse(condition)</code>	Verifica che la condizione sia false
<code>assertNull(value)</code>	Verifica che il valore sia nullo
<code>assertNotNull(value)</code>	Verifica che il valore non sia nullo
<code>assertSame(expected, current)</code>	Verifica che i <i>riferimenti</i> puntino allo <i>stesso oggetto</i>
<code>assertNotSame(expected, current)</code>	Verifica che i <i>riferimenti</i> puntino a <i>oggetti distinti</i>
<code>fail</code>	Forza il fallimento del test
....	



da **assert** a **assertXXX**

Metodo	Descrizione
assertEquals(expected, current)	Verifica che i due <i>valori</i> o <i>oggetti</i> siano <i>uguali</i>
assertArrayEquals(expected, current)	Verifica che i due <i>array</i> siano uguali
assertTrue(condition)	Verifica che la condizione sia vera
assertFalse(condition)	Verifica che la condizione sia false
assertNull(value)	Verifica che il valore sia nullo
assertNotNull(value)	Verifica che il valore non sia nullo
assertSame(expected, current)	Verifica che i <i>riferimenti</i> puntino allo <i>stesso oggetto</i>
assertNotSame(expected, current)	Verifica che i <i>riferimenti</i> puntino a <i>oggetti distinti</i>
fail	Forza il fallimento del test

.... In particolare:

- **assertEquals** nel caso di **oggetti** si basa su **equals**
- **assertSame** si basa sulla coincidenza dei riferimenti (==)



da **assert** a **assertXXX**

Metodo	Descrizione
<code>assertEquals(expected, current)</code>	Verifica che i due <i>valori</i> o <i>oggetti</i> siano <i>uguali</i>
<code>assertEquals(expected, current, delta)</code>	Verifica che i due <i>valori reali</i> siano <i>uguali</i>
<code>assertArrayEquals(expected, current)</code>	Verifica che i due <i>array</i> siano <i>uguali</i>
<code>assertTrue(condition)</code>	Verifica che la condizione sia vera
<code>assertFalse(condition)</code>	Verifica che la condizione sia false
<code>assertNull(value)</code>	Verifica che il valore sia nullo
<code>assertNotNull(value)</code>	Verifica che il valore non sia nullo
<code>assertSame(expected, current)</code>	Verifica che i <i>referimenti</i> puntino allo <i>stesso oggetto</i>

Per numeri reali, che possono essere affetti da errore:

- esiste una **assertEquals** specifica, a tre argomenti, di cui il terzo è il «delta» entro cui considerare uguali i due valori (v. oltre)

Operativamente...

- Per usare i metodi **assertXXX**, occorrerebbe ogni volta scriverne il nome completo, della forma
org.junit.jupiter.api.Assertions.assertXXX
- Poiché ciò non è pratico, conviene ricorrere ad alcune **import**
- Una prima **import «classica»** per importare i nomi delle classi (pubbliche) dell'intero package Jupiter:
 - **import org.junit.jupiter.*;**
- Una seconda **import static** per importare i nomi dei metodi statici della classe-libreria Assertions di Jupiter:
 - **import static org.junit.jupiter.api.Assertions.*;**



Un primo esempio: collaudare il Counter

- Il buon vecchio **Counter** (opportunamente «pacchettizzato»):

```
package counter;
public class Counter {
    private int val;

    public Counter() { val = 1; }
    public Counter(int v) { val = v; }

    public void reset() { val = 0; }
    public void inc() { val++; }
    public void inc(int k) { val += k; }

    public int getValue() { return val; }

    public boolean equals(Counter x) { return val==x.val; }
    public String toString(){ return "Counter di valore " + val; }
}
```

- Occorre collaudare:
 - i costruttori
 - i singoli metodi (in particolare, per ultima, la `equals`)



Collaudo del Counter (1/6)

- Come farlo?
- Per **collaudare i costruttori**, occorre:
 - costruire un oggetto `Counter` col costruttore desiderato
 - verificare che il counter abbia il valore previsto → `getValue`

NB: è impossibile collaudare un costruttore «da solo», senza accoppiarlo a un accessor di verifica → possibile vulnus (se l'accessor «mentisse»...?)

Alternativa: se il campo dati avesse *visibilità di package* si potrebbe accedervi direttamente dal test, MA al prezzo di mettere il test nello stesso package!
→ rimedio peggiore del male, comunque inopportuno

Collaudo del Counter (2/6)

- Quindi:
 - collaudiamo il costruttore a zero argomenti (almeno un test)
 - collaudiamo il costruttore a un argomento (almeno due test)
- Come lo facciamo?
 - costruendo un counter e verificando che abbia il valore previsto
 - usiamo **assertEquals** con due valori primitivi (interi)

```
package counter.test;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

import counter.Counter;

class CounterTest {

    @Test
    void testConstructor0() {
        Counter c = new Counter();
        assertEquals(1, c.getValue());
    }

    @Test
    void testConstructor1a() {
        Counter c = new Counter(10);
        assertEquals(10, c.getValue());
    }

    @Test
    void testORConstructor1b() {
        Counter c = new Counter(27);
        assertEquals(27, c.getValue());
    }
}
```

Metodo di test

Collaudo del Counter (3/6)

- Analogamente:
 - collaudiamo `reset` (un test)
 - collaudiamo `inc` (un test)
- Come lo facciamo?
 - costruendo un counter, lo resettiamo e verifichiamo che valga 0
 - costruendo un counter, lo incrementiamo (ripetutamente) e verifichiamo che il valore aumenti
 - usiamo **`assertEquals`** con due valori primitivi (interi)

```
@Test
void testReset() {
    Counter c = new Counter();
    assertEquals(1, c.getValue());
    c.reset();
    assertEquals(0, c.getValue());
}

@Test
void testInc0() {
    Counter c = new Counter();
    assertEquals(1, c.getValue());
    c.inc();
    assertEquals(2, c.getValue());
    c.inc();
    c.inc();
    assertEquals(4, c.getValue());
}
```

Collaudo del Counter (4/6)

- Analogamente:
 - collaudiamo la `inc` a un argomento (almeno due test)
- Come lo facciamo?
 - costruendo un counter, lo incrementiamo (ripetutamente, in vari modi diversi) e verifichiamo che il valore aumenti come previsto
 - usiamo **`assertEquals`** con **due valori primitivi** (interi)

```
@Test
```

```
void testIncKa() {  
    Counter c = new Counter();  
    assertEquals(1, c.getValue());  
    c.inc(1);  
    assertEquals(2, c.getValue());  
    c.inc(1);  
    c.inc(1);  
    assertEquals(4, c.getValue());  
}
```

```
@Test
```

```
void testIncKb() {  
    Counter c = new Counter();  
    assertEquals(1, c.getValue());  
    c.inc(2);  
    assertEquals(3, c.getValue());  
    c.inc(4);  
    c.inc(7);  
    assertEquals(14, c.getValue());  
}
```

Collaudo del Counter (5/6)

- Resta da collaudare `equals`
 - verifichiamo che sia riflessiva, simmetrica e transitiva
- Come lo facciamo?
 - costruendo due o più counter, poi invocando `equals` e verificando che dia *true* / *false* come previsto
 - stavolta usiamo **`assertTrue`** e **`assertFalse`**
 - utile anche un cross-check col risultato del semplice `==`

@Test

```
void testEquals() {  
    Counter c1 = new Counter(7);  
    Counter c2 = new Counter(7);  
    assertTrue(c1.equals(c2));  
    assertTrue(c2.equals(c1));  
    assertTrue(c1.equals(c1));  
}
```

@Test

```
void testNotEquals() {  
    Counter c1 = new Counter(7);  
    Counter c3 = new Counter(6);  
    assertFalse(c1.equals(c3));  
    assertFalse(c3.equals(c1));  
}
```

@Test

```
void testSame() {  
    Counter c1 = new Counter(7);  
    Counter c2 = c1;  
    assertTrue(c1==c2);  
    assertTrue(c1.equals(c2));  
}
```

Collaudo del Counter (6/6)

- DUBBIO: perché non usare il metodo **assertEquals**?
 - sarebbe fatto apposta..
- PROBLEMA: se lo facessimo, otterremmo sempre *false*! ☹️

```
c1 = new Counter(7) ;  
c2 = new Counter(7) ;  
assertEquals(c1,c2) ;
```

Fallirebbe!



- Scopriremo fra qualche lezione il vero motivo
 - in realtà, la nostra **equals** attuale ha un *difetto di fondo* che la rende *inusabile direttamente* da un framework come JUnit
 - per ora, accontentiamoci del workaround con **assertTrue**

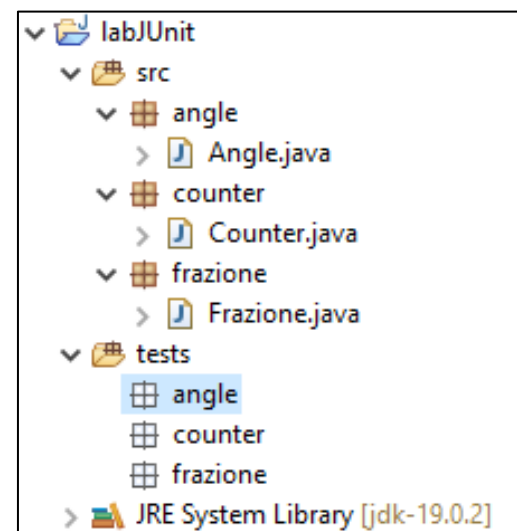
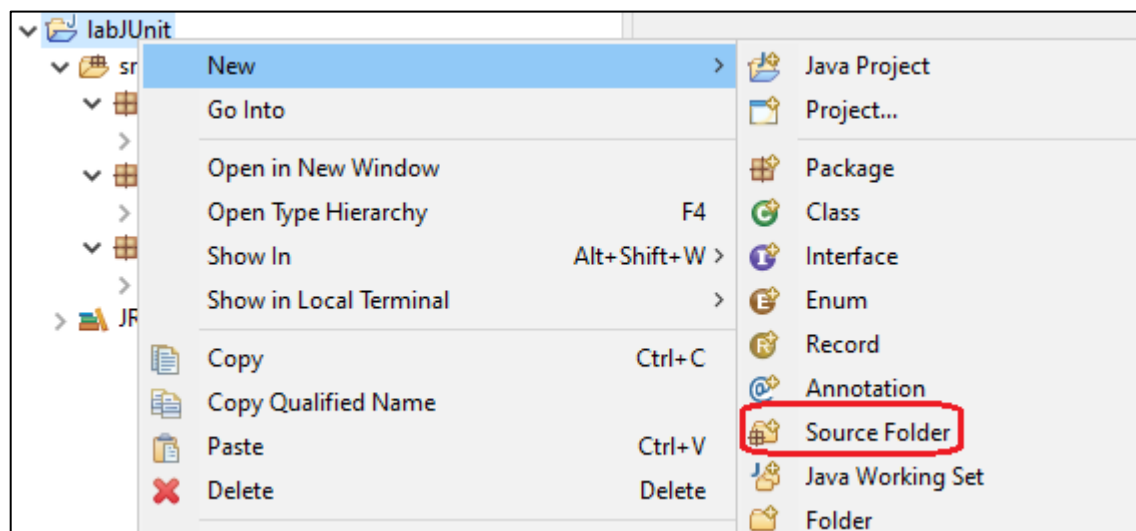


Usare più «source folder» (1/3)

- Per ***mantenere ben separati test e business logic*** è utile *non limitarsi* a mettere solo i test in *uno o più package* (cosa che, comunque, *va fatta*)
- È spesso opportuno **strutturare anche il progetto su più source folder**, così da separali anche fisicamente
- Ad esempio, potremmo affiancare alla classe source folder «**src**» una ulteriore source folder «**tests**»

Usare più «source folder» (2/3)

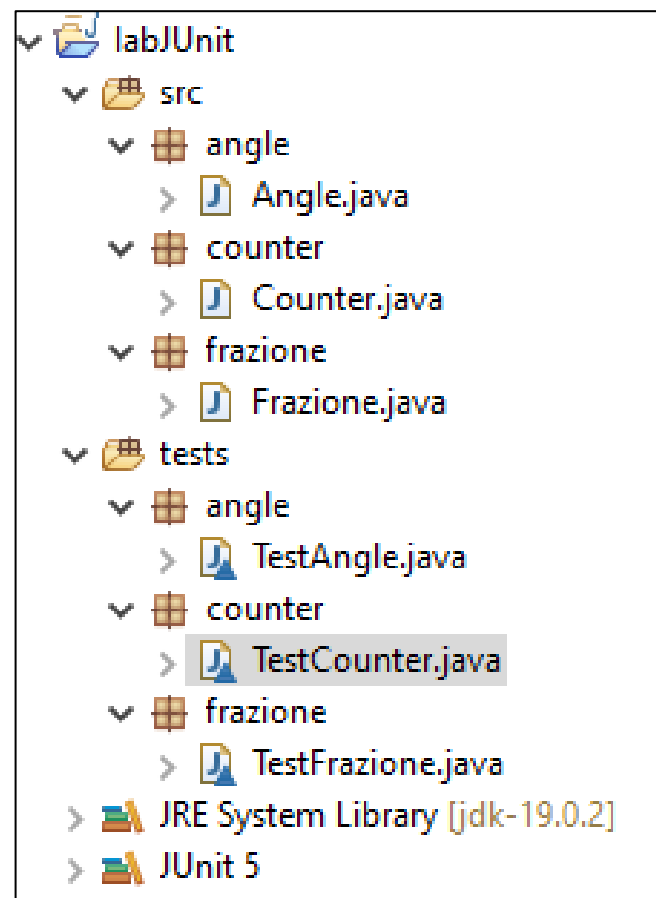
- Per aggiungere al progetto una **source folder tests**:



- Al suo interno creeremo poi i package di test (inizialmente vuoti) a cui aggiungeremo le fixture

Usare più «source folder» (3/3)

- Nel caso del collaudo di Counter (e frazioni, e angoli), avremo:
 - la **source folder tests** per i test (oltre a **src** per i sorgenti)
 - **appositi package** per sorgenti e test i cui nomi siano scelti secondo una *naming convention* logica e razionale
 - Possibile scelta: usare lo stesso naming sia per i sorgenti sia per i test (come qui: **angle**, **counter** e **frazione**)
 - Possibile alternativa migliore: *separare i naming*, introducendo ad esempio **angle.test**, **counter.test**, etc.)





Confrontare valori *floating point*

- A causa della rappresentazione dei numeri reali, l'uguaglianza fra valori reali è un concetto *approssimato*
- Di conseguenza, **assertEquals** per valori reali è diversa dalle altre, perché vale *a meno di un "delta"* da specificare
→ ha 3 argomenti anziché 2
 - il valore atteso (expected)
 - il valore corrente (actual)
 - il delta massimo (delta)ovvero *l'intorno* entro cui i due valori sono considerati “uguali”

```
double d1 = 0.1 + 0.1 + 0.1;  
double d2 = 0.3;  
double d3 = 1.0 - 0.7;
```

SEMBRANO tutti uguali...
...ma non lo sono

Confrontare valori *floating point*

- A causa della rappresentazione dei numeri reali, l'uguaglianza fra valori reali è un concetto *approssimato*
- Di conseguenza, **assertEquals** per valori reali è diversa dalle altre, perché vale *a meno di un "delta"* da specificare
→ ha 3 argomenti anziché 2
 - il valore atteso (expected)
 - il valore corrente (actual)
 - il delta massimo (delta)

E infatti...

```
jshell> double d1= 0.1 + 0.1 + 0.1;  
d1 ==> 0.30000000000000004  
  
jshell> double d2= 0.3;  
d2 ==> 0.3  
  
jshell> double d3= 1.0 - 0.7;  
d3 ==> 0.30000000000000004  
  
jshell> double d4= 0.9 - 0.6;  
d4 ==> 0.30000000000000004  
  
jshell> double d5= 0.8 - 0.5;  
d5 ==> 0.30000000000000004  
  
jshell> double d6= 0.5 - 0.2;  
d6 ==> 0.3
```



Altri collaudi: frazioni, angoli

- Allo stesso modo si possono convertire in test JUnit i collaudi «artigianali» a suo tempo studiati per
 - **Frazione**
 - **Angle**
- **MA occhio ai valori reali negli angoli!!!**

Ciò è lasciato per esercizio al lettore ☺



Un passo in più

Strutturazione dei test

- È frequente che più test richiedano *operazioni comuni*
 - prima e dopo l'esecuzione di *tutti i test di una intera classe*
 - prima e dopo l'esecuzione di *ogni singolo test*
- A tal fine conviene:
 - incapsulare in opportuni metodi quelle operazioni comuni
 - etichettare tali metodi per farli riconoscere al Runner di JUnit

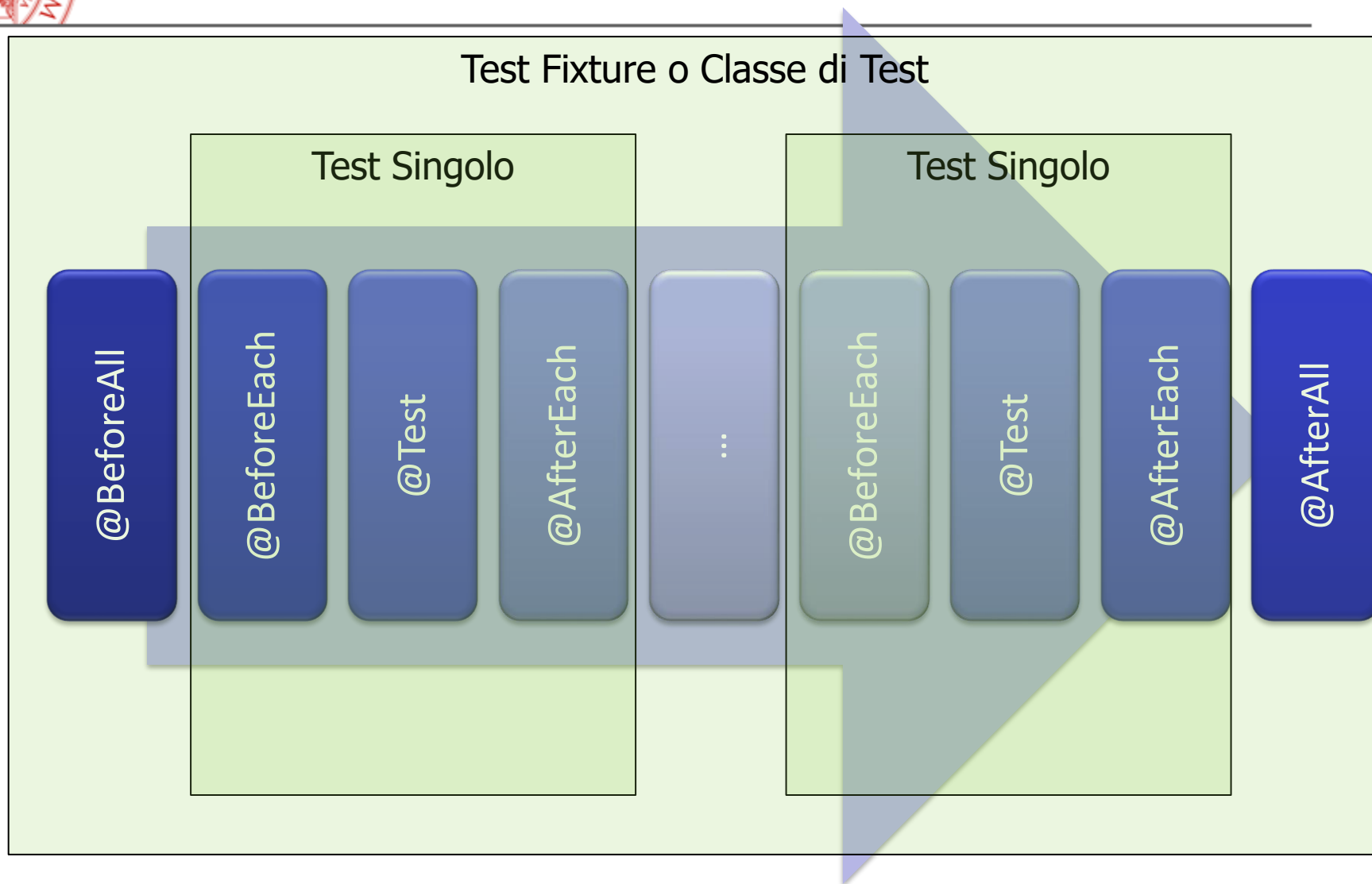
- **Annotation disponibili:**

- **@BeforeAll / @AfterAll:** marca una singola funzione statica da eseguire **UNA SOLA VOLTA** prima/dopo **l'intera classe di test**
- **@BeforeEach / @AfterEach:** marca un metodo (non statico) da eseguire **OGNI VOLTA** prima/dopo **ogni singolo metodo di test**

Statica perché riguarda l'intera classe,
non il singolo oggetto-test

Non statico perché riguarda
il singolo oggetto-test

Sequenza di esecuzione





Esempio: MyCalendar (1/4)

Ad esempio, nell'applicazione «Calendario» (che presto realizzerete in laboratorio), tre test hanno un'**operazione iniziale in comune**:

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;

public class MyCalendarTest {
    MyCalendar myCal;

    @Test
    public void testAdd() {
        myCal = new MyCalendar();
        LocalDateTime from = LocalDateTime.of(2019, Month.MARCH, 10, 12, 30, 0);
        LocalDateTime to = LocalDateTime.of(2019, Month.MARCH, 10, 15, 30, 0);
        Appointment app = new Appointment("Compleanno", from, to);
        myCal.add(app);
        assertEquals(1, myCal.getAllAppointments().size());
    }
    ...
}
```

Esempio: MyCalendar (2/4)

Ad esempio, nell'applicazione «Calendario» (che presto realizzerete in laboratorio), tre test hanno un'**operazione iniziale in comune**:

@Test

```
public void testGetListAppointmentDay() {  
    myCal = new MyCalendar();  
    LocalDateTime from1 = LocalDateTime.of(2019, Month.MARCH, 10, 12, 30, 0);  
    LocalDateTime to1 = LocalDateTime.of(2019, Month.MARCH, 10, 15, 30, 0);  
    Appointment app1 = new Appointment("Compleanno", from1, to1);  
    myCal.add(app1);  
    LocalDateTime from3 = LocalDateTime.of(2019, Month.MARCH, 10, 20, 30, 0);  
    LocalDateTime to3 = LocalDateTime.of(2019, Month.MARCH, 10, 22, 30, 0);  
    Appointment app3 = new Appointment("Cena Lavoro", from3, to3);  
    myCal.add(app3);  
    LocalDate date = LocalDate.of(2019, Month.MARCH, 10);  
    AppointmentCollection coll = myCal.getDayAppointments(date);  
    assertEquals(2, coll.size());  
}
```



Esempio: MyCalendar (3/4)

Ad esempio, nell'applicazione «Calendario» (che presto realizzerete in laboratorio), tre test hanno un'**operazione iniziale in comune**:

@Test

```
public void testRemove() {  
    myCal = new MyCalendar();  
    LocalDateTime from1 = LocalDateTime.of(2019, Month.MARCH, 10, 12, 30, 0);  
    LocalDateTime to1 = LocalDateTime.of(2019, Month.MARCH, 10, 15, 30, 0);  
    Appointment app1 = new Appointment("Compleanno", from1, to1);  
    myCal.add(app1);  
    LocalDateTime from3 = LocalDateTime.of(2019, Month.MARCH, 10, 20, 30, 0);  
    LocalDateTime to3 = LocalDateTime.of(2019, Month.MARCH, 10, 22, 30, 0);  
    Appointment app3 = new Appointment("Cena Lavoro", from3, to3);  
    myCal.add(app3);  
    assertTrue(myCal.remove(app3));  
    LocalDate date = LocalDate.of(2019, Month.MARCH, 10);  
    assertEquals(1, myCal.getDayAppointments(date).size());  
}
```



Esempio: MyCalendar (4/4)

Ha quindi senso fattorizzare tale **operazione iniziale in comune** in un metodo da eseguire OGNI VOLTA, PRIMA della fixture

→ tag **@BeforeEach**

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;

public class MyCalendarTest {
    MyCalendar myCal;

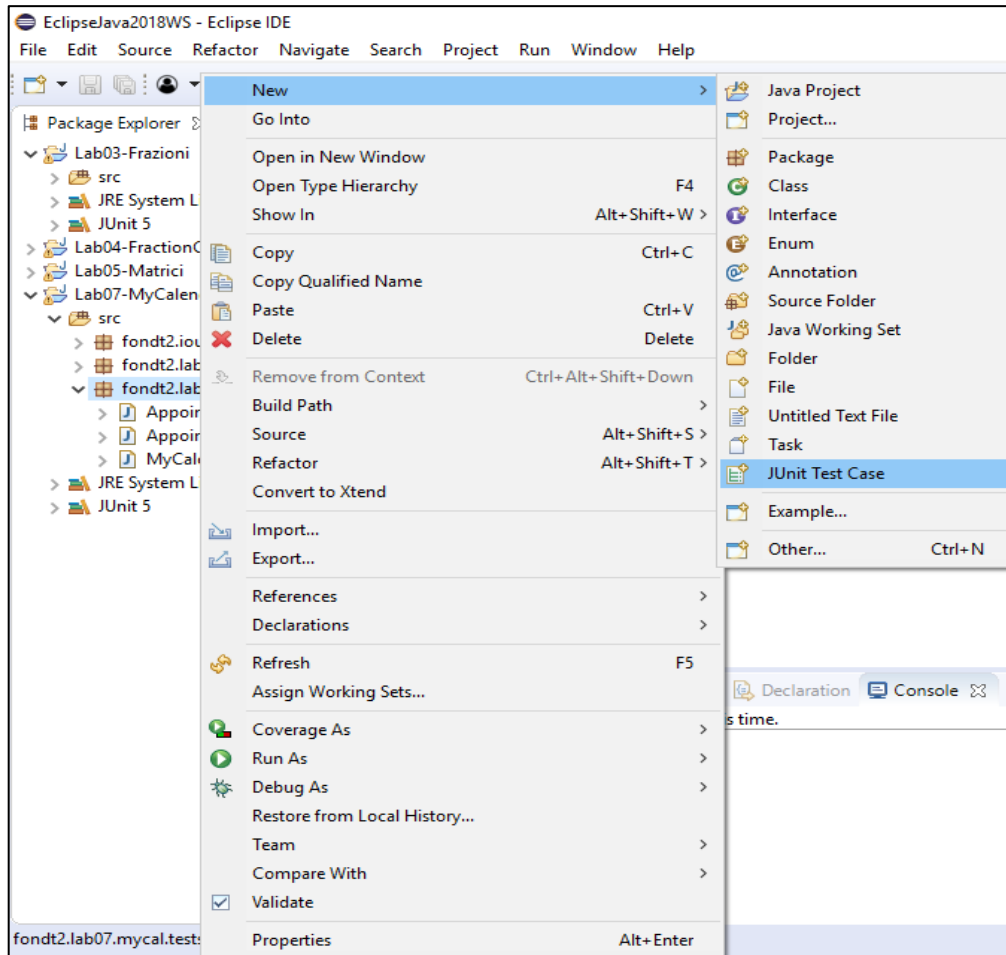
    @BeforeEach
    public void setUp() {
        myCal = new MyCalendar();
    }
}
```

La corrispondente operazione può quindi ora essere **rimossa** dal codice dei singoli test

VANTAGGIO: eventuali modifiche/integrazioni sono **centralizzate**

Integrazione con Eclipse (1/6)

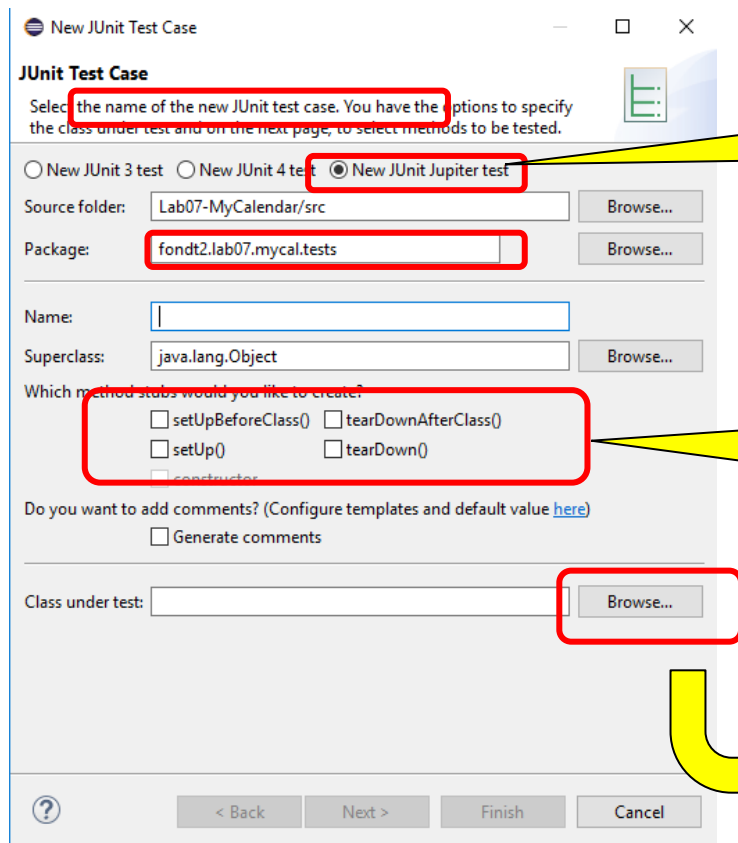
- Eclipse facilita la creazione dei test tramite *wizard*



Tasto destro sulla classe
da collaudare
New > JUnit Test Case

Integrazione con Eclipse (2/6)

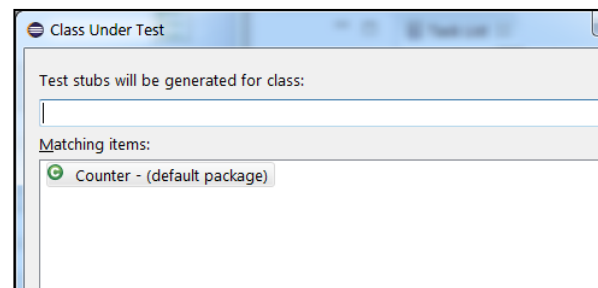
- Il wizard permette di scegliere i metodi da collaudare e crea lo scheletro della Fixture di test.



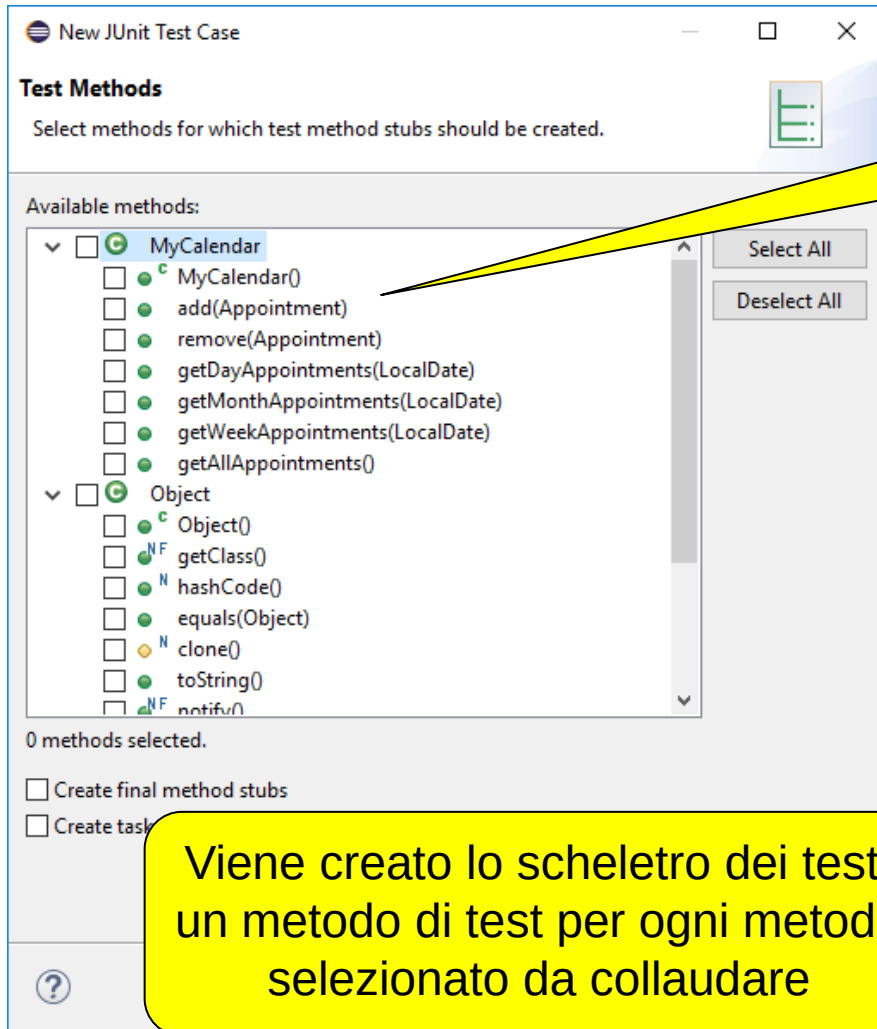
Importante impostare JUnit 5

- la sintassi di JUnit 4 era diversa

Genera lo scheletro dei due metodi **@AfterEach**, **@BeforeEach** e delle due funzioni statiche **@AfterAll**, **@BeforeAll**



Integrazione con Eclipse (3/6)



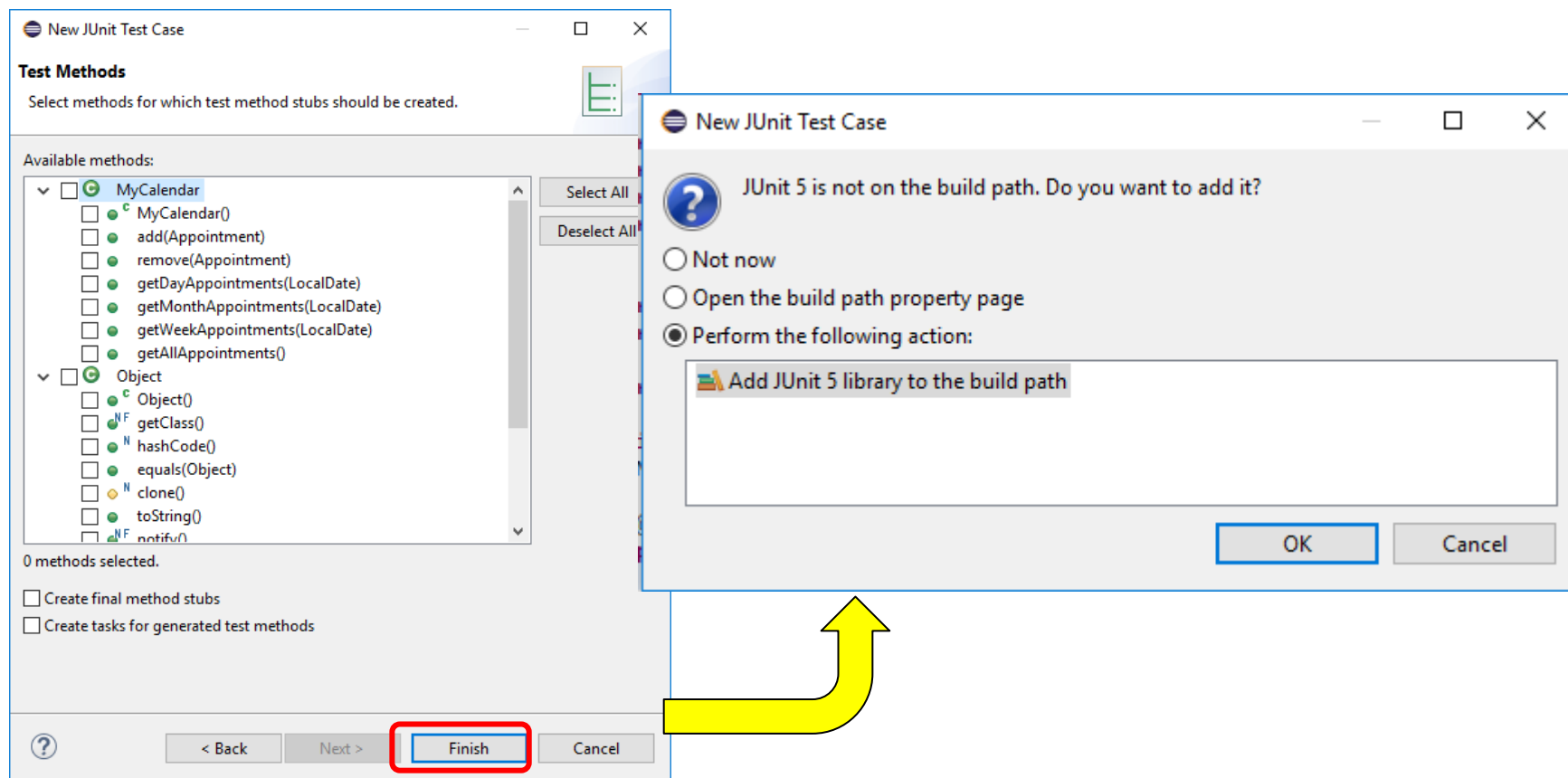
Selezione dei metodi di cui si vogliono effettuare i test

Viene creato lo scheletro dei test:
un metodo di test per ogni metodo
selezionato da collaudare

```
1 package fondt2.lab07.mycal.tests;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5
6
7
8
9
10
11 class ProvaTest {
12
13     @BeforeAll
14     static void setUpBeforeClass() throws Exception {
15     }
16
17     @AfterAll
18     static void tearDownAfterClass() throws Exception {
19     }
20
21     @BeforeEach
22     void setUp() throws Exception {
23     }
24
25     @AfterEach
26     void tearDown() throws Exception {
27     }
28
29     @Test
30     void testMyCalendar() {
31         fail("Not yet implemented");
32     }
33
34     @Test
```

Integrazione con Eclipse (4/6)

- Se non già presente, viene chiesto di aggiungere JUnit al build path → sì, certo!



Integrazione con Eclipse (5/6)

The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure, including the package `fondt2.lab07.mycal.tests` which contains the `ProvaTest.java` file. The main editor displays the code of `ProvaTest.java`, which includes static methods for class setup and teardown, and instance methods for test setup and teardown, along with several test methods.

```
11 class ProvaTest {
12
13     @BeforeAll
14     static void setUpBeforeClass() throws Exception {
15     }
16
17     @AfterAll
18     static void tearDownAfterClass() throws Exception {
19     }
20
21     @BeforeEach
22     void setUp() throws Exception {
23     }
24
25     @AfterEach
26     void tearDown() throws Exception {
27     }
28
29     @Test
30     void testMyCalendar() {
31         fail("Not yet implemented");
32     }
33
34     @Test
35     void testAdd() {
36         fail("Not yet implemented");
37     }
38
39     @Test
40     void testRemove() {
41         fail("Not yet implemented");
42     }
43
44     @Test
```

Three yellow callout boxes highlight specific parts of the code:

- Scheletro delle due funzioni statiche @AfterAll, @BeforeAll**: Points to the static methods `setUpBeforeClass()` and `tearDownAfterClass()`.
- Scheletro dei due metodi @AfterEach, @BeforeEach**: Points to the instance methods `setUp()` and `tearDown()`.
- Scheletro dei vari metodi di test**: Points to the test methods `testMyCalendar()`, `testAdd()`, and `testRemove()`.

The right-hand side of the IDE shows the Task List and a list of methods in the `ProvaTest` class, including `setUpBeforeClass()`, `tearDownAfterClass()`, `setUp()`, `tearDown()`, and various `test` methods.

Integrazione con Eclipse (6/6)

Verde per i test passati

Blu (Failure) per i falliti causa attese non verificate (fallimento di un'asserzione)

Rosso (Error) per i falliti causa «esplosione» imprevista (es. utilizzo di riferimento nullo)

Run as → JUnit Test
sulla fixture che si vuole eseguire

EclipseJava2018WS - Lab07-MyCalendar/src/fondt2/lab07/mycal/tests/ProvaTest.java - Eclipse IDE

File Edit Source Refa

Package Explorer

5.SafeRepo

6.Tris

6.Tris-StartKit

Lab03-Frazioni

Lab04-FractionColl

Lab05-Matrici

Lab07-MyCalendar

src

fondt2.ioutils

fondt2.lab07

fondt2.lab07

Appointm

Appointm

MyCalend

ProvaTest

JRE System Libr

JUnit 5

New

Open F3

Open With

Open Type Hierarchy F4

Show In Alt+Shift+W

Copy Ctrl+C

UpBeforeClass() throws Exception {

Declarations

Refresh F5

Assign Working Sets...

Coverage As

Run As

Debug As

Restore from Local History...

Team

Compare With

JUnit Test Alt+Shift+X, T

Run Configurations...

Task List

Quick Access

Find

All Activate...

Package Explorer JUnit

Finished after 0,328 seconds

Runs: 5/5 Errors: 0 Failures: 0

MyCalendarTest [Runner: JUnit 5] (0,000 s)

01/02/2019 11:47



... e se qualcosa non va?

- Se i test falliscono, non è sufficiente «prenderne atto»
- Occorre capire *perché* falliscono
- JUnit dice esattamente cosa sia accaduto:
 - BLU = asserzione fallita
→ indica quale e perché
 - ROSSO = errore ("esplosione") nell'esecuzione del test
→ riporta il messaggio d'errore e lo *stack trace*
(elenco dei record di attivazione)



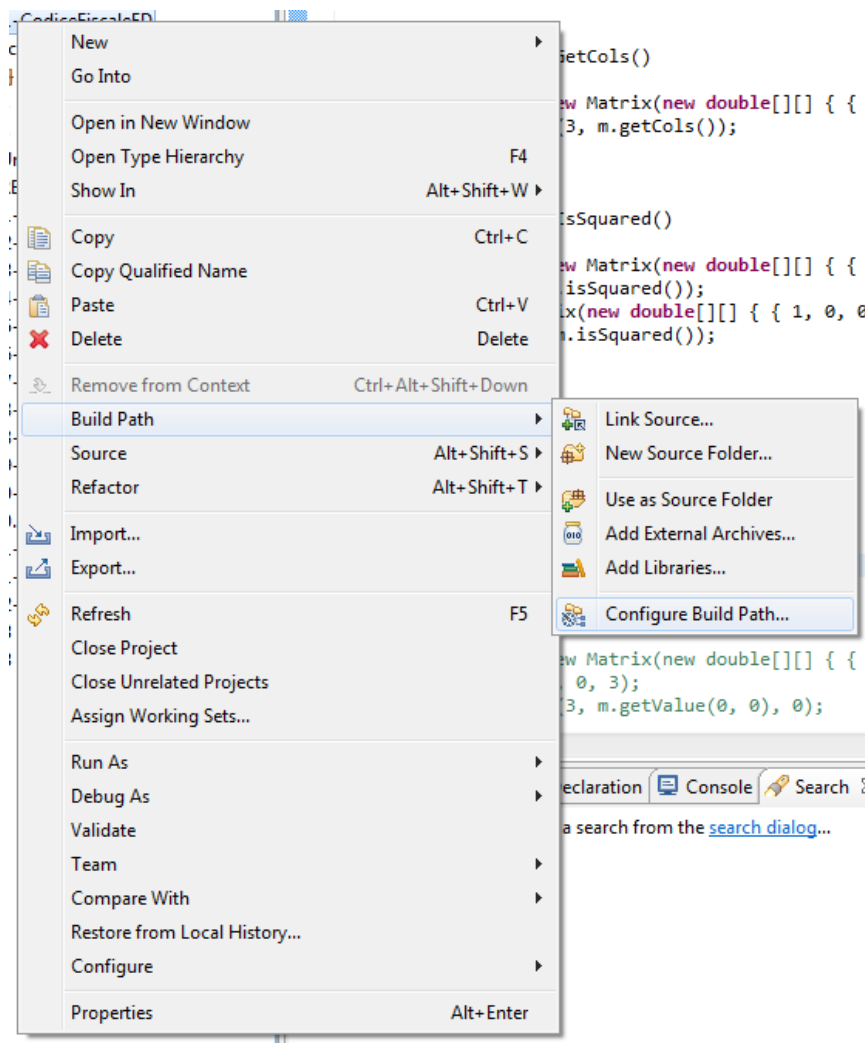
Quando Eclipse non collabora

- Tecnicamente, JUnit è una normale libreria (JAR)
→ il corrispondente JAR dev'essere nel progetto
- Come abbiamo visto, di norma creando un nuovo *JUnit Test case*, il JAR di JUnit viene norma aggiunto al build path *automaticamente*
- Tuttavia, qualche volta l'automatismo non scatta
→ **può essere necessario aggiungere JUnit al progetto manualmente**

Aggiungere JUnit al progetto (1/2)

PROCEDURA

1. Selezionare il progetto col tasto destro
2. Selezionare ora la voce *Build Path*
3. Selezionare quindi il menù *Configure Build Path...*



Aggiungere JUnit al progetto (2/2)

Nella finestra che appare:

1. Andare nella scheda *Libraries*
2. Premere il pulsante *Add Library*
3. Scegliere *JUnit 5* e confermare

